

Character Classifier Inspector Tool

https://github.com/ldoughty05/character_classifier

By Luke Doughty

Introduction

I have been interested in machine learning, and especially deep learning for the past two years and I learned the basics from online and with a textbook called Grokking Deep Learning by Andrew W. Trask. This gave me the basic intuition on neural neural networks and how they learn using gradient descent. The textbook walked me through building my own neural network framework to solve simple problems like character classification and text generation. I felt the natural next step would be to use an established framework like Pytorch to make more sophisticated models so I can focus my learning on understanding network architectures.

About the Model

I decided to build a character classification model using the MNIST dataset because it is a common problem and there are hundreds of different approaches each with recorded accuracies and number of parameters for me to benchmark the quality of my own model. My model also works with the Extended MNIST (EMNIST) dataset which includes hand written numbers as well as upper and lower case letters.

I managed to get 94.17% accuracy with MNIST (numbers only) and 86.5% accuracy with EMNIST (numbers and letters) using the model shown below. It starts with a convolution where it breaks the image up into multiple smaller images. The convolutional layer effectively checks for smaller patterns like a vertical line, a horizontal line, or a diagonal line. The flatten method turns those subimages into a vector which can be plugged into the following Linear layer. The linear layer learns to correlate the presence or absence of features found by the convoluted layer with a feature of a number or letter. The output of this linear layer then gets squeezed using the Tanh activation function so that the range is between -1 and 1. The output of tanh is then put through another linear layer where it correlates the features of the drawing with a number or a letter. For example, the convoluted layer may notice a circle and a line on the bottom right side. The linear layers would see that and say that those two shapes together make a 9.

The dropout is used to silence 50% of the nodes in the linear layer so that one node doesn't learn to overpower all the others, or so that the network doesn't learn to always associate some small feature with a certain output. A vertical line doesn't always correlate with a 1. Silencing certain neurons allows the model to consider the possibility of other features correlating with certain outputs instead of hyperfixating on a handful of features.

```
In [ ]: self.sequential_network = torch.Sequential(
    torch.Conv2d(1, 1, 3, bias=False),
    # the 3x3 convolution reduces the number of pixels by 2, so the convolutio
    torch.Flatten(), # turns the 2d image output of the conv2d into a vector
    torch.Linear(get_convolution_output_size(28, 28, 3), 800, bias=False),
    torch.Tanh(), # squeezes values to be between -1 and 1
    torch.Dropout(p=0.5), # prevents co-adaptation of feature detectors preve
    torch.Linear(800, get_num_classes(dataset), bias=False),
)
# Learning Rate: 0.02
# Trained for 5 epochs
```

The diagram below shows the method calls to train a new neural network. Using a framework like Pytorch dramatically reduces the amount of code to make the model itself especially since backpropagation is handled automatically. Almost every method takes in either `dataset_name` or `model_name`. This was necessary because these methods need to be able to know which model they are working with so they can get information like input or output size. This stinks like a DRY violation. I would like to instead specify the model and dataset once at the beginning of the training process and not have to think about it in the following method calls. Using a global variable is a possible solution, but it may make it unclear to other people working with this code where the current value of the global variable comes from. A global variable would be a poor solution at least from a readability and coupling standpoint.

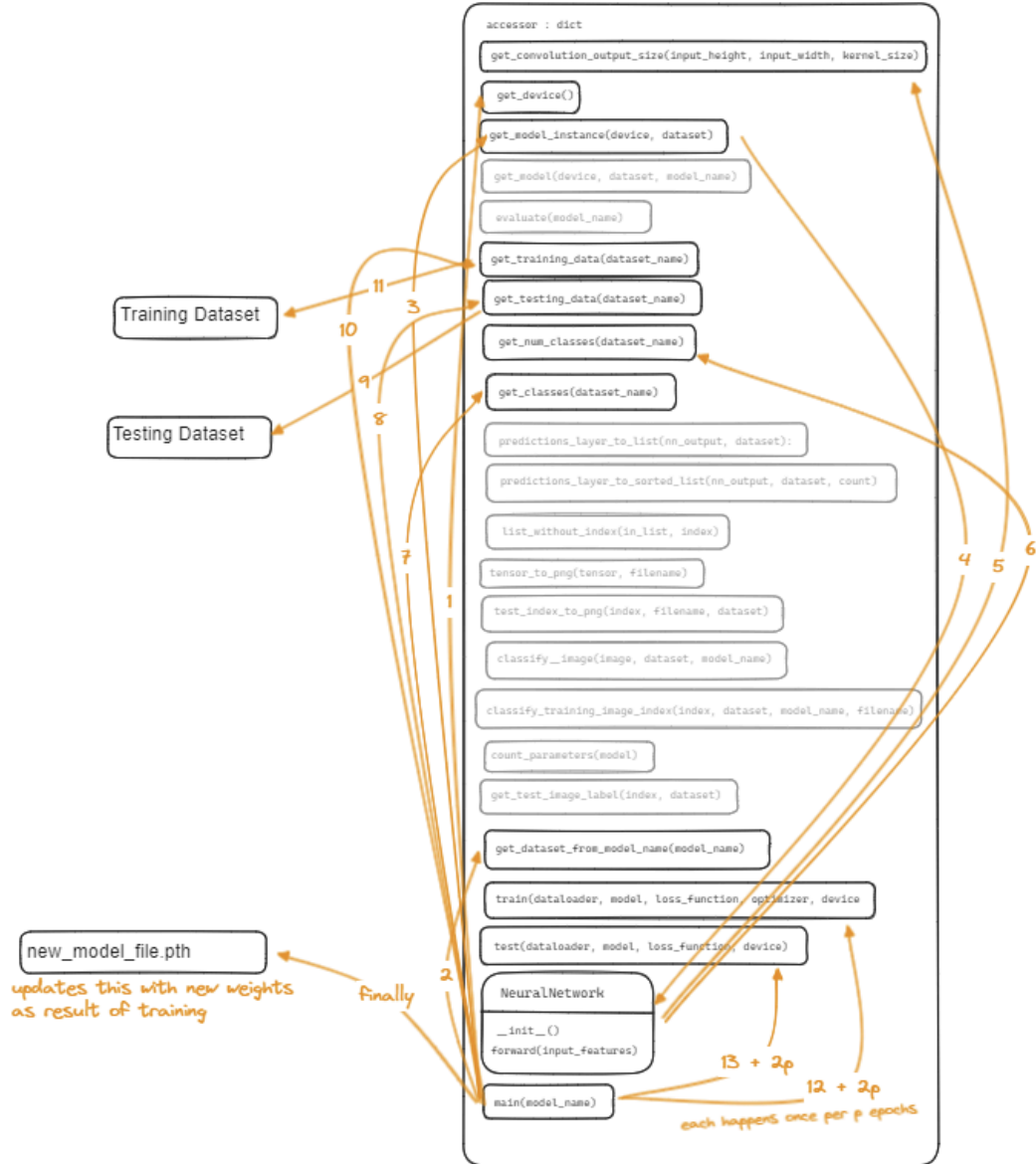
To move canvas, hold mouse wheel or spacebar while dragging, or use the hand tool



Running classifier as `__name__ == "__main__"` while `TRAINING_NEW_MODEL == True`

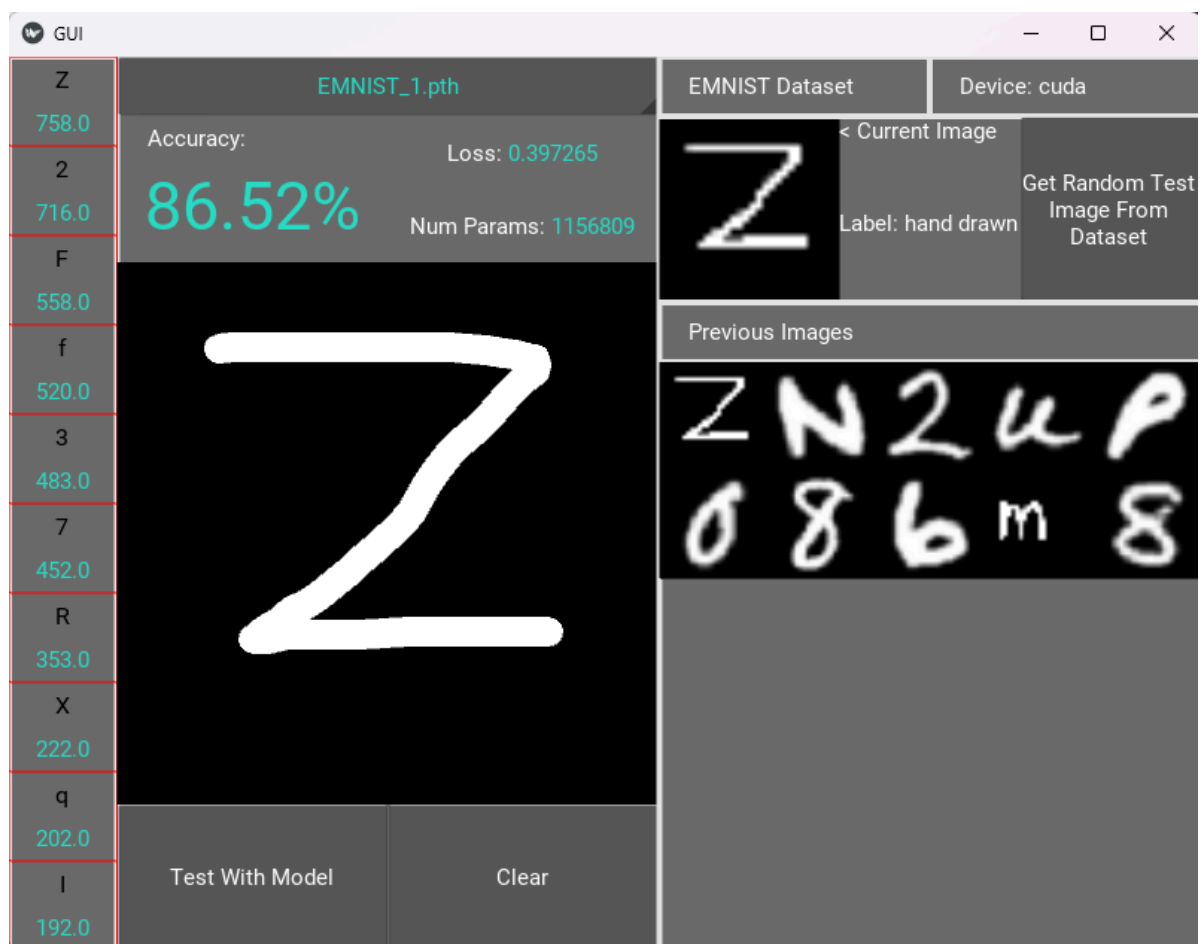
Back End

classifier.py



About the GUI

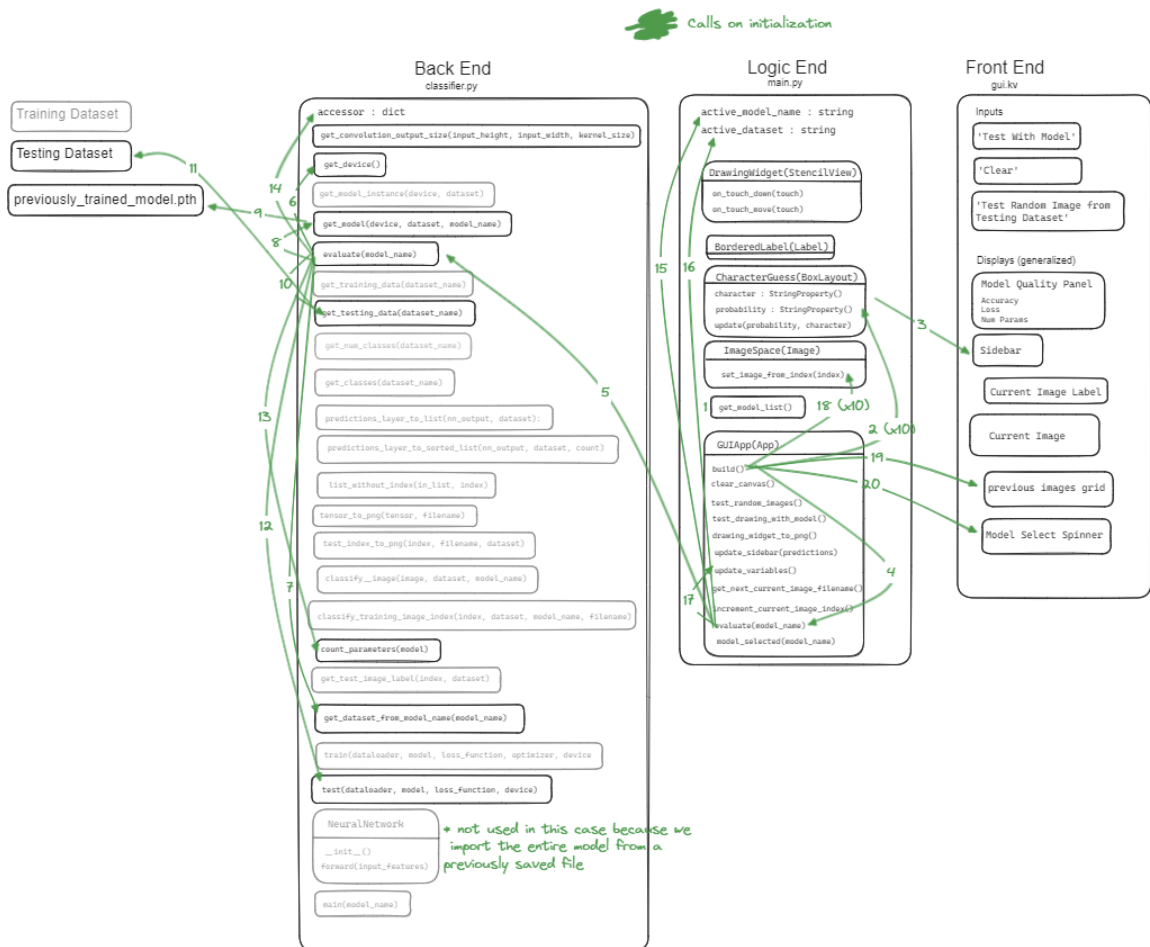
The GUI uses the kivy library to display information about the neural network. On startup and when a model is selected, it runs the entire testing dataset through the neural network to determine the accuracy and average loss of the model, as well as counting the number of trainable parameters. Those statistics are an easy indicator of the functional quality of a model. There is a button that pulls a random image from the testing dataset and runs it through the network. The bar on the far left side shows the output classes (what number/letter it is) and the probability score of that class. The output classes are displayed in order of highest probability on top, so the topmost number should match the image label. The ten most recent test images are displayed below the random image button. Other information like the current dataset, name of the current model, and the device (cpu or cuda) are displayed at the top of the window.



App Startup

Anything with deep learning is going to involve lots of matrix multiplication which is very computationally expensive. This is noticeable in the time it takes for the program to start. As shown in the diagram below, there are lots of processes that need to happen before the app is ready to take on user input. The most taxing method calls are probably `test()`, `get_dataset_from_model_name()`, and `get_testing_data()`. Test involves multiplying very large

matrices so naturally that will take a long time since it has to do hundreds of multiplication operations. The other two methods contribute to the long startup times because they involve accessing data from the computer's storage device. The program has to retrieve the entire training dataset which is quite large (EMNIST especially). I would like to learn more about accessing large amounts of data from storage more efficiently and in smaller chunks, as well as how the data is stored in memory after it has been retrieved for the sake of reducing load times in my future projects.



Three Layer Architecture

The front end does not interact with the neural network directly. There is a logic layer in between that gets whatever information is needed from the neural network in the backend and processes it for the front end to display. A three layer architecture helps with encapsulation and keeping the code readable, maintainable, and minimizes coupling. A drawback was that I often found myself making redundant calls to get information in the back end even though it has already been calculated in the logic layer or in another back end method. An example being getting the active device or getting the correct dataset for each model. Often, a value gets calculated in some backend method, but it might not be something I can store in the logic layer or between event instances so the only option is to make those redundant calls to get the necessary information. If I were to do this again I would find a better way to store the active

model and device with some getter method that simply accesses a stored value rather than recalculating that same value.

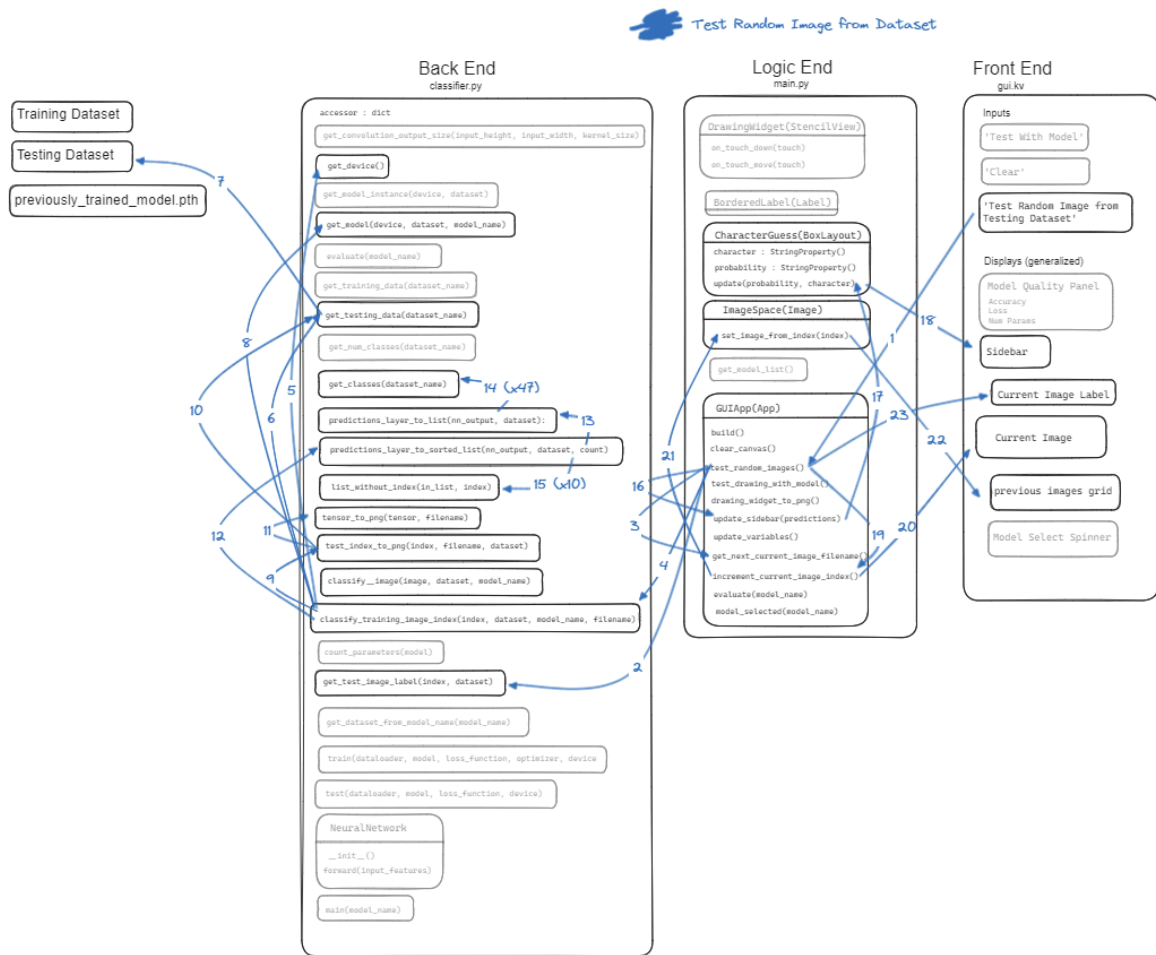
Going from images to tensors

Working with neural networks means working with tensors which were very hard to work with at first because they are hard to mentally visualize. The images in the training data were stored as 3D tensors, so converting them to a png to show in my kivy gui was a challenge. I used the `torch.transforms.toPILImage()` method to convert tensors to something that can be saved as a png. To turn my drawing widget into a tensor, I used the `cv2` library to resize the image to the required 28x28 pixels and turn it to greyscale. The `torch.imread()` method then turned the modified image into a tensor. I then had to use the `torch.unsqueeze(1)` method to add another dimension to the tensor to specify the number of channels (an RGB image would have three channels, my images only have one for greyscale) as required by my convolutional layer.

```
In [ ]: # METHODS USED TO CLASSIFY IMAGE FROM DRAWING WIDGET
def drawing_widget_to_png(self): # Starts here in the logic layer to turn my drawing w
    # Get drawing from widget
    drawing_widget = self.root.ids.drawing_widget
    file = f"user_made_char.png"
    drawing_widget.export_to_png(file)
    # format the widget png to a 28x28 greyscale
    test_image = cv2.imread(file, cv2.IMREAD_GRAYSCALE)
    image_resized = cv2.resize(test_image, (28, 28), interpolation=cv2.INTER_LINEAR)
    formatted_image = self.get_next_current_image_filename()
    cv2.imwrite(formatted_image, image_resized)
    return formatted_image # returns the filepath, not the image itself

def classify_image(image, dataset, model_name): # backend method to convert the test i
    device = get_device()
    model = get_model(device, dataset, model_name)
    image_tensor = read_image(image, ImageReadMode.GRAY)
    image_tensor = image_tensor.unsqueeze(1)
    image_tensor = image_tensor.float()
    with torch.no_grad():
        image_tensor = image_tensor.to(device)
        predictions = model(image_tensor)
    return predictions_layer_to_sorted_list(predictions, dataset) # this list is sent
```

Testing random images from the training data had a similar process, but it is less complicated because the logic layer can get data from the back end by simply sending the index of the test image. The back end can then get the image tensor from the training database, feed it through the neural network, and then it returns the test image label, sorted predictions list, and the png of the training image back to the logic layer to be displayed on the gui.



Model Selector Dropdown

This was the last piece of functionality I added, and also the most difficult to implement. Much of the backend code was reliant on the model, datasets, and active device being static and initialized when the app is started, so making that variable meant lots of refactoring. This also led to the most frustrating bug in the whole project development. For some reason the model started giving completely wrong predictions, and it had very low confidence in all of its predictions. What happened was that I was making a new instance of the neural network every time i wanted to make a prediction, and I had forgotten to import the trained weight values each time. I added the code to import the weights into my `get_model()` method, but I do not like that implementation because making a new instance of the model and importing the weights every time is certainly redundant. I could instead save the model in the logic layer or save it in the back end as a global variable, but both of those solutions are bad practices.

Final Thoughts

This project taught me the importance of making call graphs during the development process to help spot redundancy and unnecessary coupling early on. I also learned to always use version control even on personal projects. I had given up on using version control at the start of the project because I had problems with setting up my python environment in pycharm and I kept

having to make new project files. Not using git was regrettable because I often made changes that broke things and I had no way to look back at working versions of my code. This project helped me recognize tradeoffs in design decisions. While performance is very important, it sometimes comes at the cost of increased memory usage or lower encapsulation which can impact maintainability and even security. After having to consider those tradeoffs in my own work, I am more thoughtful in my engineering decision making.

I feel I accomplished my goal of becoming fluent in building neural networks using a framework. I gained the intuition to be more intentional with the way I iterate on a model. The next step in my learning is to try to replicate more sophisticated models described in research papers and trying to understand why they structured their model the way they did. Another avenue for improvement would be to look into transformers and other attention mechanisms.